

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

LUCAS APARECIDO DOS SANTOS

**LAR DOCE LAR: JOGO DE SOBREVIVÊNCIA ZUMBI
DESENVOLVIDO EM C++ COM RAYLIB**

**CAMPOS DO JORDÃO
2026**

RESUMO

O presente relatório apresenta o desenvolvimento do jogo digital Lar Doce Lar, um jogo 2D de sobrevivência zumbi com visão superior, desenvolvido em linguagem C++ com o auxílio da biblioteca gráfica raylib. O jogo coloca o jogador no papel de um sobrevivente que precisa defender uma casa contra invasões de zumbis, utilizando recursos coletados durante o dia para reparar e reforçar janelas, além de enfrentar inimigos com ataques corpo a corpo. A proposta combina mecânicas de exploração, defesa de base, gerenciamento de recursos, progressão de dificuldade e combate em tempo real.

O projeto foi estruturado com base nos princípios da programação orientada a objetos, separando responsabilidades em classes específicas, essa organização permitiu maior clareza no desenvolvimento, melhor reaproveitamento de código e separação entre lógica, desenho, colisões, entidades e controle geral do jogo. Como resultado, foi obtido um jogo funcional, com ciclo de dia e noite, sistema de itens, inventário de armas, ataques carregados, zumbis com comportamento próprio, HUD informativo, sprites aplicados e regras de interação com janelas e barricadas. O trabalho demonstra a aplicação prática de conceitos de programação, lógica de jogos digitais e organização modular de software.

Palavras-chave: jogo digital; C++; raylib; programação orientada a objetos; sobrevivência zumbi.

ABSTRACT

This report presents the development of the digital game *Lar Doce Lar*, a top-down 2D zombie survival game developed in C++ using the raylib graphics library. The game places the player in the role of a survivor who must defend a house against zombie invasions, using resources collected during the day to repair and reinforce windows, as well as confront enemies with melee attacks. The game combines exploration mechanics, base defense, resource management, progressive difficulty, and real-time combat.

The project was structured based on the principles of object-oriented programming, separating responsibilities into specific classes; this organization allowed for greater clarity in development, better code reuse, and separation between logic, graphics, collisions, entities, and general game control. As a result, a functional game was produced, featuring a day-night cycle, an item system, a weapon inventory, charged attacks, zombies with unique behavior, an informative HUD, applied sprites, and interaction rules for windows and barricades. The work demonstrates the practical application of programming concepts, digital game logic, and modular software organization.

Keywords: digital game; C++; raylib; object-oriented programming; zombie survival.

SUMÁRIO

1	INTRODUÇÃO _____	12
1.1	Objetivos _____	6
1.2	Justificativa _____	6
1.3	Aspectos Metodológicos _____	7
1.4	Aporte Teórico _____	7
2	METODOLOGIA _____	8
3	RESULTADOS OBTIDOS _____	10
4	CONCLUSÃO _____	13
	REFERÊNCIAS _____	15

1 INTRODUÇÃO

Os jogos digitais constituem uma área importante de aplicação da programação, pois exigem integração entre lógica, entrada do usuário, atualização de estados, renderização gráfica, colisões, controle de tempo e organização de recursos. Nesse contexto, o projeto Lar Doce Lar foi desenvolvido com o objetivo de aplicar conceitos de programação orientada a objetos na criação de um jogo 2D funcional e jogável, utilizando a linguagem C++ e a biblioteca raylib.

O jogo pertence ao gênero de sobrevivência e defesa de base. Sua narrativa apresenta um sobrevivente em meio a um cenário de apocalipse zumbi, no qual a casa representa o último local seguro. Durante o dia, o jogador pode se movimentar pelo mapa e coletar recursos, como madeira e armas. Durante a noite, a ameaça dos zumbis se intensifica, exigindo que o jogador proteja a casa, repare janelas danificadas e utilize ataques para se defender. A proposta do jogo está centrada na tensão entre preparação e sobrevivência: quanto melhor o jogador aproveita o período diurno, maiores são suas chances de resistir aos ataques durante a noite.

1.1 Objetivos

O objetivo geral do projeto foi desenvolver um jogo 2D de sobrevivência zumbi, com mecânicas de movimentação, combate, coleta, reparo, barricadas, ciclo de dia e noite, HUD e comportamento básico de inimigos. Como objetivos específicos, destacam-se: implementar uma arquitetura orientada a objetos; criar uma classe central responsável por coordenar o mundo do jogo; desenvolver entidades com vida, posição e colisão; implementar zumbis capazes de perseguir o jogador e atacar janelas; criar um sistema de coleta de itens; construir um sistema de ataques com carga e durabilidade de armas; aplicar sprites ao ambiente e às entidades; e organizar a interface visual com informações úteis ao jogador.

1.2 Justificativa

A justificativa do projeto está relacionada à possibilidade de unir teoria e prática no estudo de programação. Em vez de desenvolver apenas exemplos isolados, o jogo

exige que diferentes conceitos sejam combinados em um sistema único. A criação de Lar Doce Lar permitiu aplicar herança, encapsulamento, composição de objetos, listas dinâmicas, funções auxiliares, controle de tempo por delta time, colisões geométricas, gerenciamento de recursos gráficos e separação de responsabilidades entre classes. Assim, o trabalho demonstra não apenas o funcionamento de um jogo, mas também a importância de uma arquitetura organizada para manter o código legível e expansível.

1.3 Aspectos Metodológicos

Quanto aos aspectos metodológicos, o projeto foi desenvolvido por meio de uma abordagem prática e aplicada. A implementação foi organizada em etapas: definição da ideia central do jogo, criação das classes principais, desenvolvimento da movimentação do jogador, implementação da casa e das colisões, criação dos zumbis, adição dos itens, construção do sistema de ataque, implementação do ciclo de dia e noite, aplicação dos sprites e organização final do HUD. O desenvolvimento foi guiado por testes constantes de jogabilidade, verificando se as regras estavam funcionando de forma coerente.

1.4 Aporte Teórico

O aporte teórico do trabalho está baseado nos conceitos de programação orientada a objetos e desenvolvimento de jogos 2D. A programação orientada a objetos foi utilizada para representar os elementos do jogo como entidades com responsabilidades próprias. A biblioteca raylib forneceu os recursos de janela, desenho, teclado, tempo, texturas e renderização. A playlist de tutoriais sobre raylib serviu como apoio prático para compreender a estrutura básica de jogos com essa biblioteca, principalmente no uso de loop principal, funções de desenho, entrada de teclado, atualização por frame e organização de elementos gráficos.

2 METODOLOGIA

O desenvolvimento de Lar Doce Lar foi realizado em C++, utilizando a biblioteca raylib como principal ferramenta gráfica. O ambiente de desenvolvimento utilizado foi o Visual Studio Code, com compilação por meio de MinGW/G++ em sistema Windows. O projeto foi organizado em arquivos de cabeçalho e implementação, separando as classes em arquivos .hpp e .cpp, o que contribuiu para uma estrutura mais limpa e compreensível.

A metodologia adotada pode ser classificada como aplicada, pois o objetivo foi construir um produto jogável a partir dos conhecimentos adquiridos em programação. Também possui caráter descritivo, pois o relatório apresenta o funcionamento do sistema, as decisões de desenvolvimento e os resultados obtidos. O desenvolvimento seguiu uma lógica incremental: primeiro foram implementadas as estruturas essenciais do jogo e, em seguida, foram adicionadas novas mecânicas e melhorias visuais.

A estrutura principal do jogo foi organizada em torno da classe `Mundo`, responsável por coordenar os principais elementos da aplicação. O arquivo `main.cpp` inicia a janela do jogo, define a taxa de quadros e cria um objeto da classe `Mundo`. A cada frame, o programa calcula o tempo decorrido, atualiza as regras do jogo e realiza o desenho da cena. Essa organização mantém o `main.cpp` simples e concentra a lógica principal em uma classe própria.

A classe `Entidade` foi criada como base para elementos vivos do jogo, reunindo atributos comuns, como posição, raio de colisão, vida, vida máxima e estado vivo ou morto. A partir dela, foram derivadas as classes `Jogador` e `Zumbi`, que compartilham características gerais, mas possuem comportamentos diferentes. Essa escolha demonstra o uso de herança e reaproveitamento de código, evitando repetição desnecessária.

A classe `Jogador` controla a movimentação, a vida, a direção do olhar, o carregamento de ataques, o cooldown, o inventário de armas e a exibição de informações relacionadas ao personagem. O jogador pode se mover com as teclas W, A, S, D ou com as setas direcionais. O ataque é realizado ao segurar e soltar a tecla espaço, permitindo golpes leves, médios ou fortes conforme o tempo de carregamento. O sistema de armas inclui as mãos como recurso padrão, além de armas coletáveis, como taco de beisebol e cano de metal. As armas possuem dano, alcance e durabilidade, sendo que a durabilidade só é reduzida quando o ataque realmente acerta um zumbi.

A classe `Zumbi` representa os inimigos do jogo. Os zumbis possuem vida, velocidade, dano e comportamento de perseguição. Quando o jogador está fora da casa, os zumbis o perseguem diretamente. Quando o jogador está dentro da casa, os zumbis que estão fora procuram a janela mais próxima. Se a janela estiver inteira, eles a atacam; se estiver quebrada, atravessam a abertura e continuam a perseguição. Essa lógica cria uma dinâmica de invasão coerente com a proposta do jogo.

A classe `Casa` é responsável por representar a base do jogador. Ela controla o chão, as paredes, a escada, as janelas, as colisões, os reparos e as barricadas. A casa possui seis janelas distribuídas pelas paredes. Janelas inteiras funcionam como barreiras, enquanto janelas quebradas se tornam passagens. O jogador pode reparar janelas usando madeira ou criar barricadas para aumentar sua resistência. O jogo também impede que o jogador repare ou instale uma barricada em uma janela quando está sobreposto a ela, evitando situações em que o personagem poderia ficar preso.

A classe `Item` representa os objetos coletáveis do mapa. Durante o dia, são gerados itens fora da área da casa, incluindo madeira, taco de beisebol e cano de metal. A madeira funciona como recurso para reparos e barricadas. As armas são adicionadas ao inventário do jogador, respeitando o limite de uma arma equipada e até três armas reservas.

A classe `TexturasJogo` centraliza o carregamento e descarregamento dos sprites utilizados no jogo. Essa decisão evita carregar imagens repetidamente durante o desenho dos frames, o que prejudicaria o desempenho. As texturas são carregadas uma vez no início e descarregadas ao final da execução. Entre os sprites aplicados estão o jogador, zumbi vivo, zumbi morto, grama, chão da casa, parede, escada, janelas, barricada, madeira, taco, cano de metal e mãos.

A classe `Utilidades` reúne funções matemáticas e de colisão utilizadas em diferentes partes do projeto. Entre essas funções estão cálculo de distância, normalização de vetores, limitação de valores, colisão entre círculo e retângulo, e verificação de linhas contra retângulos. Essa separação evita que cálculos comuns fiquem repetidos em várias classes.

O jogo também possui um sistema de ciclo de dia e noite. Durante o dia, o jogador pode coletar itens e preparar a casa. Durante a noite, os zumbis se tornam mais perigosos e aparecem com maior intensidade. A dificuldade aumenta progressivamente conforme os dias avançam, fazendo com que a sobrevivência se torne mais desafiadora ao longo da partida.

3 RESULTADOS OBTIDOS

O resultado obtido foi um jogo 2D funcional de sobrevivência zumbi, com mecânicas integradas de movimentação, combate, coleta, defesa de base, ciclo temporal, progressão de dificuldade, HUD e sprites.

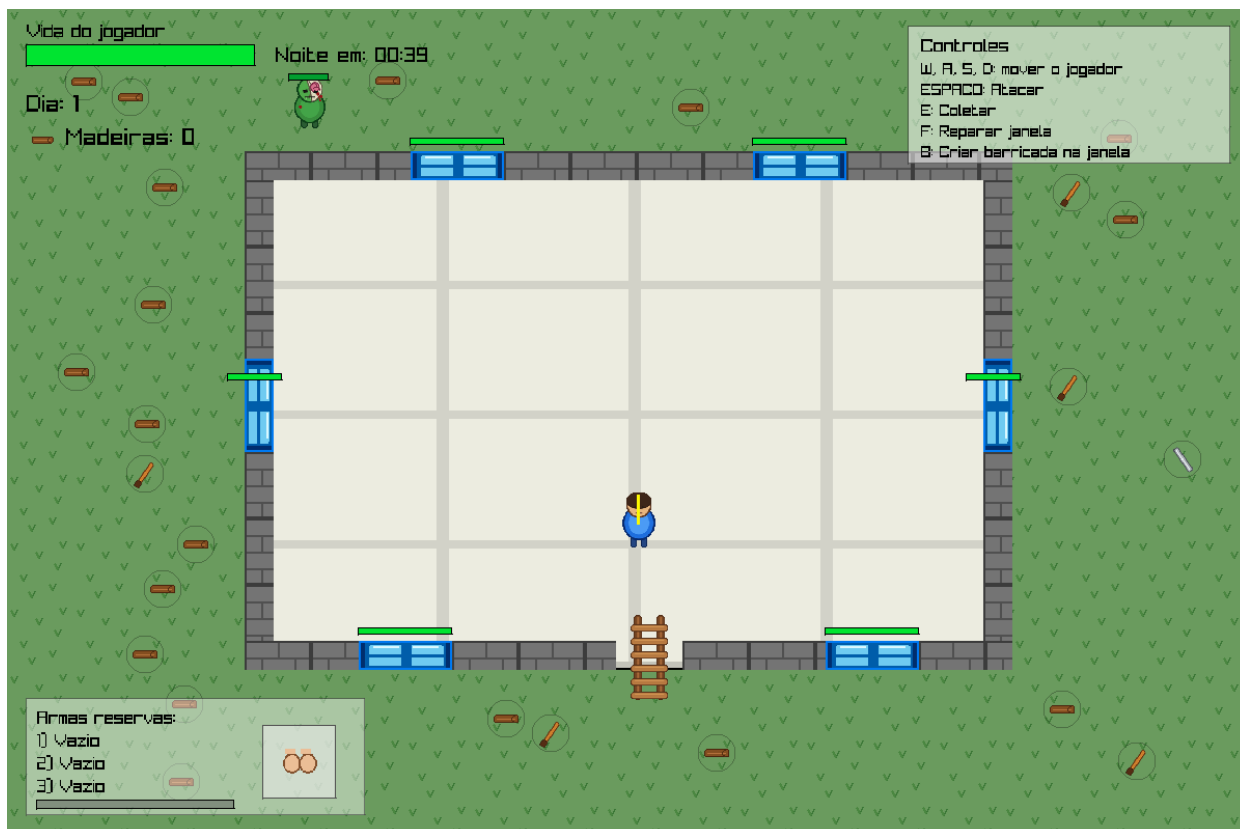


Figura 1 – Tela principal do jogo com jogador, casa, HUD e zumbi no canto superior esquerdo

A Figura 1 apresenta a visão geral do jogo Lar Doce Lar. Nela é possível observar a câmera em perspectiva superior, o ambiente externo, a casa central, o personagem controlado pelo jogador e as informações exibidas no HUD. O canto superior esquerdo mostra dados como vida, dia atual, tempo restante para a troca de período e quantidade de madeiras. Também está presente nessa Figura os itens gerados aleatoriamente no exterior da casa, no qual o jogador pode coletar e usar esses recursos para defender sua casa, sendo criando barricadas nas janelas ou usando como armas brancas para eliminar zumbis.



Figura 2 – Noite, sistema de janelas e armas

A Figura 2 apresenta o jogo quando anoitece e os zumbis se tornam mais fortes, rápidos e em maior número, a imagem mostra 3 janelas quebradas, 2 janelas reforçadas com barricadas e 1 janela ainda inteira. No canto inferior esquerdo, na HUD é possível observar que o jogador está equipado com a arma “Taco de beisebol” e possui mais duas dessa mesma arma em seu inventário como armas reservas a serem usadas quando a equipada se desgastar até seu limite, o desgaste da arma só é contabilizado quando um zumbi é acertado para não punir o jogador por errar um ataque ou acertar a parede. Também demonstra o momento de maior tensão do jogo: a noite. Nesse período, os zumbis aparecem com maior intensidade e se tornam mais ameaçadores. Caso o jogador esteja dentro da casa, eles procuram a janela mais próxima para atacar ou atravessar, caso ela já esteja quebrada. Essa lógica cria um desafio progressivo, pois o jogador precisa equilibrar combate direto, manutenção da casa e uso de recursos.



Figura 3 – Ataque do jogador e zumbi morto

A figura 3 ilustra o jogador atacando usando a tecla ESPAÇO e sua área de ataque, os zumbis que estiverem dentro dessa área circular destacada em amarelo receberá o dano correspondente a arma equipada e força do ataque, caso a vida do zumbi seja igual ou menor a zero ele é eliminado.

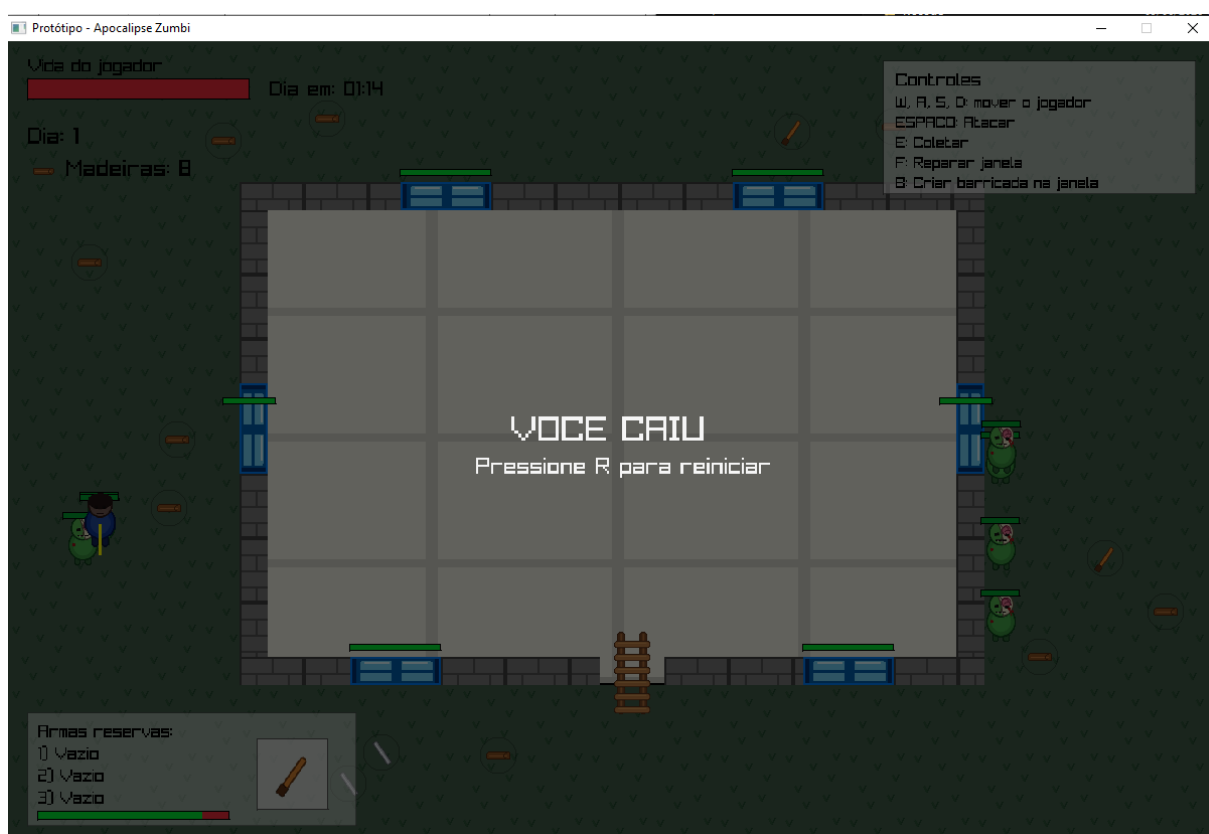


Figura 4 – Tela de derrota do jogador

A figura 4 apresenta a tela exibida quando o jogador perde toda a vida. O jogo informa a derrota e permite reiniciar a partida com a tecla R. Esse recurso fecha o

ciclo básico de jogabilidade, permitindo que o jogador tente novamente e busque sobreviver por mais dias.

De modo geral, os resultados demonstram que o jogo alcançou seus objetivos principais. A arquitetura orientada a objetos possibilitou a separação das responsabilidades, o sistema de combate funcionou integrado às colisões, os zumbis apresentaram comportamento coerente com a proposta, os itens foram incorporados à progressão do jogador e o HUD forneceu informações essenciais durante a partida.

4 CONCLUSÃO

O desenvolvimento de Lar Doce Lar permitiu aplicar, de forma prática, conceitos fundamentais de programação orientada a objetos e desenvolvimento de jogos digitais. O projeto resultou em um jogo 2D simples, com mecânicas de sobrevivência, defesa de base, coleta de recursos, combate, progressão de dificuldade e interface visual informativa.

A principal contribuição do trabalho foi demonstrar como diferentes classes podem atuar em conjunto para formar um sistema jogável. A classe `Mundo` centraliza a coordenação geral; `Jogador` controla as ações do personagem; `Zumbi` define o comportamento dos inimigos; `Casa` gerencia colisões, janelas e barricadas; `Item` representa recursos coletáveis; `Entidade` evita repetição de atributos comuns; `TexturasJogo` organiza os sprites; e `Utilidades` reúne funções matemáticas reutilizáveis. Essa separação tornou o código mais organizado, compreensível e adequado para manutenção.

O jogo também se destaca por apresentar regras consistentes de interação. Janelas inteiras bloqueiam passagem, janelas quebradas permitem invasão, barricadas fortalecem a defesa e armas só perdem durabilidade quando acertam um zumbi. Essas decisões contribuem para uma jogabilidade mais justa e estratégica, pois o jogador precisa pensar tanto na movimentação quanto no uso dos recursos disponíveis.

Como sugestões de melhorias futuras, podem ser adicionados efeitos sonoros para ataques, coleta, dano e quebra de janelas; animações de sprites; menu inicial; tela de pausa; sistema de pontuação; salvamento de recorde de dias sobrevividos; diferentes tipos de zumbis; troca manual de arma equipada; armas de longo alcance com munição limitada; sistema de raridade para itens; e melhoria no comportamento dos inimigos por meio de *pathfinding* mais avançado. Também seria possível aprimorar o balanceamento da dificuldade, criando curvas de progressão mais detalhadas conforme o avanço dos dias.

REFERÊNCIAS

YOUTUBE. Raylib Tutorials. Playlist de tutoriais utilizada como apoio no estudo e aplicação dos recursos da biblioteca raylib. Disponível em: <https://www.youtube.com/playlist?list=PLwR6ZGPvjVOSRywn9VCQ3yrRVruxzzuo9>. Acesso em: 01/06/2026.